DISCRETE MATHEMATICS

1. This question carries 30% of the marks.

   a) For each of the following statements, state whether it is true or false and provide a supporting proof.

      i) $2n+1 = O(n)$. [3]

      Answer:
      True, since $2n+1 \leq 3n$ for all $n \geq 1$.

      ii) $n+(-1)^n n = \Theta(n)$. [3]

      Answer:
      False. For odd n, the LHS is 0 and this is not $\geq cn$ for any $c > 0$.

      iii) $n^2 = \omega(n(n+1))$ [3]

      Answer:
      False. The ratio of LHS to RHS is $n^2/n(n+1) = n/(n+1) \to 1$ as $n \to \infty$ (would need to be ∞ for ω).

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

      i) $T(n) = 9T(n/3) + n$. [2]

      Answer:
      $a = 9, b = 3, d = 1$ so $\log_b a = \log_3 9 = 2 > d = 1$ so $T(n) = O(n^{\log_b a}) = O(n^2)$.

      ii) $T(n) = 4T(n/4) + n^3$. [2]

      Answer:
      $a = 4, b = 4, d = 3$ so $\log_b a = \log_4 4 = 1 < 3 = d$ so $T(n) = O(n^d) = O(n^3)$.

      iii) $T(n) = 8T(n/4) + O(n^2)$. [2]

      Answer:
      $a = 8, b = 4, d = 2$ so $\log_b a = \log_4 8 = 1.5 < 2 = d$ so $T(n) = O(n^d) = O(n^2)$.

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Answers

2. This question carries 20% of the marks.

The quicksort algorithm is defined as follows:

```
function QUICKSORT(A)
    q ← PARTITION(A)
    QUICKSORT(A[1 ··· q − 1])
    QUICKSORT(A[q + 1 ··· n])
end function
function PARTITION(A)
    p ← A[n]
    i ← 0
    for j = 1 to n − 1 do
        if A[j] ≤ p then
            i ← i + 1
            swap A[i] and A[j]
        end if
    end for
    swap A[n] and A[i + 1]
    return i + 1
end function
```

It works by choosing a "pivot" element p (the last element of the array in the implementation above), and iterating through the list to ensure that all values of A less than p are to the left of p and all values greater than p are to the right. The array is then split into two parts, and the algorithm is called recursively on each part.

a) Write down a tight bound using Θ notation for the time complexity of the PARTITION function. [1]

Answer:

The PARTITION function does $n-1$ comparisons and swaps, so $T(n) = \Theta(n)$.

b) Write down a recurrence relation for $T(n)$, the time complexity of the quicksort algorithm, in terms of the location q of the pivot element. [1]

Answer:

$T(n) = T(q-1) + T(n-q) + O(n)$.

c) What is the worst case time complexity of the quicksort algorithm? [2]

Answer:

The worst case is when $q = 1$ or $q = n$, so $T(n) = T(n-1) + O(n)$ which is $O(n^2)$.

d) What is the best case time complexity of the quicksort algorithm? [2]

Answer:

The best case is when $q = n/2$, so $T(n) = 2T(n/2) + O(n)$ which, applying the master method, is $O(n \log n)$.

e) In this part we will compute the average case time complexity. Let $C(n)$ be the average number of comparisons made by the quicksort algorithm on an array of length n.

i) Write down a recurrence relation for $C(n)$ in terms of the location q of the pivot element. [1]

Answer:

$C(n) = n - 1 + C(q-1) + C(n-q)$.

Answers

ii) Assume each location of the pivot element is equally likely. Show that

$$C(n) = n - 1 + \frac{2}{n}\sum_{i=0}^{n-1} C(i).$$

[1]

Answer:
Each location of the pivot element is equally likely, so $C(n) = n-1+\frac{1}{n}\sum_{i=1}^{n} C(i-1)+C(n-i)$. This simplifies to $C(n) = n-1+\frac{2}{n}\sum_{i=0}^{n-1} C(i)$.

iii) By calculating $nC(n)-(n-1)C(n-1)$, show that

$$\frac{C(n)}{n+1} \leq \frac{C(n-1)}{n} + \frac{2}{n+1}.$$

[1]

Answer:
We have $nC(n)-(n-1)C(n-1) = n(n-1)+2\sum_{i=0}^{n-1} C(i)-(n-1)(n-2)-2\sum_{i=0}^{n-2} C(i)$. Subtracting common terms and simplifying we get $nC(n) = (n+1)C(n-1)+2n-2$. Dividing by $n(n+1)$ gives

$$\frac{C(n)}{n+1} = \frac{C(n-1)}{n} + \frac{2}{n+1} - \frac{2}{n(n+1)} \leq \frac{C(n-1)}{n} + \frac{2}{n+1}.$$

iv) Show that $C(n) = O(n\log n)$.

Hint: use the fact that $\sum_{i=1}^{n-1} \frac{1}{i} \approx \log n$. [1]

Answer:
Repeatedly applying the previous inequality gives

$$\frac{C(n)}{n+1} \leq 2\sum_{i=1}^{n-1} \frac{1}{i}.$$

Using the hint, we have $\sum_{i=1}^{n-1} \frac{1}{i} \approx \log n$ so $C(n) \leq 2(n+1)\log n = O(n\log n)$.

Answers

3. This question carries 50% of the marks.

Consider the following pseudocode designed to run on a single processor:

```
function CACHED
    for i = 1 to m do
        x ← X_i
        Y_i ← f(x)
    end for
    for j = 1 to n do
        i ← h(j) and 1 ≤ i ≤ m
        y ← Y_i
        Z_j ← g(y)
    end for
end function
```

The computation of the array Z is broken down into two parts: first an intermediate array Y of length m is computed, and then this array is re-used in the computation of the array Z of length n. Assume that the time taken to do one read and one write to memory is t_m, that each computation f or g takes time t_c, and that all other operations take negligible time. Assume also that $m = \alpha n$ for some constant $0 < \alpha < 1/2$.

a) Compute the time taken to run this pseudocode, assuming all other operations take negligible time. [4]

Answer:

The first part does m reads and m writes, for a total of mt_m. The second part does n reads and n writes for a total of nt_m. So the total time spent reading and writing memory is $(n+m)t_m = (\alpha+1)nt_m$. The total time spent computing is similarly $mt_c + nt_c = \alpha nt_c + nt_c = (\alpha+1)nt_c$. So the total time is $(\alpha+1)nt_m + (\alpha+1)nt_c = (\alpha+1)n(t_m+t_c)$.

b) Write a parallel version of this algorithm that still computes the intermediate array Y. You may use the "parallel for" and "sync" keywords. [2]

Answer:

```
function CACHED
    for i = 1 to m in parallel do
        x ← X_i
        Y_i ← f(x)
    end for
    sync
    for j = 1 to n in parallel do
        i ← h(j)
        y ← Y_i
        Z_j ← g(y)
    end for
end function
```

c) Assume that computation can be done in parallel, but that reads and writes to the arrays X, Y and Z in global memory are queued in series. In other words, if thread 1 and thread 2 both read from global memory at the same time, one thread will have to wait for the other thread's read to complete. Compute the work T_1 and span T_∞ of your parallel algorithm, ignoring the time taken to launch threads. [4]

Answer:

The work T_1 is the same as the previous part, $T_1 = (\alpha+1)n(t_m+t_c)$. The span of

Answers

the first parallel for loop will be the sum of the spans of the memory operations and computation operations. The memory operations are carried out in series, and therefore their span will be the same as in the computation of the work, i.e. mt_m. The computations are carried out in parallel, so their span will be t_c. The span of the first loop is therefore $mt_m + t_c$. Similarly, the span of the second loop is $nt_m + t_c$ and so the span of the whole algorithm is $(\alpha+1)nt_m + 2t_c$.

d) Write a parallel version of this algorithm (named RECOMPUTE) that computes the array Z directly, without computing Y. [4]

Answer:

Each thread j computes the intermediate y value it needs, which may mean this computation is repeated multiple times.

```
function RECOMPUTE
    for j = 1 to n in parallel do
        i ← h(j)
        x ← X_i
        Z_j ← g(f(x))
    end for
end function
```

e) Compute the work T_1 and span T_∞ of your parallel algorithm in the previous part. [4]

Answer:

To compute the work, we note that we need to do n reads and writes for a total of nt_m and we need to compute f and g for each of n threads, for a total of $2nt_c$. So $T_1 = nt_m + 2nt_c$.

To compute the span, the read and write operations will be the same nt_m. The computations will be in parallel, so just $2t_c$. The span is therefore $T_\infty = nt_m + 2t_c$.

f) On a single processor system, which algorithm should you use if $t_m = t_c$? [1]

Answer:

On a single processor, we need to compare T_1 for both algorithms. The CACHED algorithm has $T_1 = (\alpha+1)n(t_m + t_c)$ and the RECOMPUTE algorithm has $T_1 = nt_m + 2nt_c$. We should use CACHED if $(\alpha+1)n(t_m + t_c) < nt_m + 2nt_c$. Writing $t_c = t_m = t$, this simplifies to $(\alpha+1)n(2t) < 3nt$ or $\alpha < 1/2$ which is stated in the question. We should therefore always use the CACHED algorithm.

g) On a system with an infinite number of processors, which algorithm should you use? [1]

Answer:

With infinite processors, we need to compare T_∞ for both algorithms. The CACHED algorithm has $T_\infty = (\alpha+1)nt_m + 2t_c$ and the RECOMPUTE algorithm has $T_\infty = nt_m + 2t_c$. Since the t_c terms are the same for both, but the t_m term is larger for the CACHED algorithm, we should always use the RECOMPUTE algorithm.

h) Consider a system with p processors. Assume threads are scheduled with a greedy scheduler and that this scheduler has worst case performance $T_p = T_1/p + T_\infty$. For which values of p should you use the CACHED and RECOMPUTE algorithms when $t_m = t_c$? [5]

Answer:

We know that for $p = 1$ and $p = \infty$ we should use CACHED or RECOMPUTE respectively, so we just need to compute the switching point where T_p is the same for both algorithms. For the CACHED algorithm, $T_p = (\alpha+1)n(t_m + t_c)/p + (\alpha+1)nt_m + 2t_c$. For the RECOMPUTE algorithm, $T_p = nt_m/p + 2nt_c + nt_m +$

Answers

$2t_c$. Setting these equal gives $t_m((\alpha+1)n(1/p+1))+t_c((\alpha+1)n/p+2) = t_m(n(1/p+1))+t_c(2n/p+2)$. Putting the t_m and t_c terms on different sides gives $t_m\alpha n(1/p+1) = t_c(1-\alpha)n/p$. Multiplying by p/n gives $t_m\alpha(1+p) = t_c(1-\alpha)$. Since $t_m = t_c$ we can simplify to $p = 1/\alpha - 2$. So, when $p < 1/\alpha - 2$ or $p = 1$ we should use CACHED, otherwise we should use RECOMPUTE.

Answers